



Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Estado de México

Implementación manual de Random Forest para clasificación binaria y multiclase.

Inteligencia Artificial avanzada para la ciencia de datos

Roberd Otoniel Meza Sainz

A01801490

Grupo 601

Introducción y objetivo

El objetivo de este proyecto es implementar un algoritmo de clasificación visto en el módulo sin utilizar frameworks de machine learning o estadística avanzada. Se eligió Random Forest, construido sobre una implementación propia de árbol de decisión como componente base.

Se utilizaron las dependencias numpy y pandas para tanto los arreglos como para la lectura y manipulación de los .csv, sin usarlos para implementar el algoritmo en sí.

Se probó la implementación en cuatro datasets distintos, en orden creciente de dificultad/tamaño:

- Iris (binario: versicolor vs virginica): validación inicial en un problema simple y linealmente casi separable.
- Breast Cancer Wisconsin (Diagnostico): problema binario real de tamaño mediano (569 muestras aprox, 30 features).
- Wine: problema multiclase de 3 clases para probar que el árbol y el bosque generalizan más allá del caso binario.
- MAGIC Gamma Telescope: dataset binario "grande" (19000 muestras aproximadamente), para evaluar desempeño y tiempo de entrenamiento con más datos.

Algoritmo

La implementación se organiza en tres clases y varias funciones auxiliares. A continuación, se describe qué hace cada pieza del código.

Clase `TreeNode`

Representa un nodo de un árbol, ya sea de decisión o de una hoja, contiene lo siguiente:

- `feature` y `threshold`: la columna y el valor de corte que usa el nodo para decidir hacia dónde mandar una muestra (izquierda si `feature <= threshold`, derecha si no).
- `left` y `right`: referencias a los nodos hijos.
- `valor`: solo se llena si el nodo es una hoja, y contiene la clase que predice ese nodo.

El método `es_hoja()` solo revisa si `valor` no es `None` distinguiendo así un nodo de decisión de una hoja al recorrer el árbol.

Clase `DecisionTree`

Función `gini(y)`: recorre las etiquetas `y` de un nodo y calcula qué tan mezcladas están las clases, con la fórmula de impureza de Gini:

```
gini = 1.0 - np.sum(probabilidades**2)
```

Donde p_i es la proporción de la clase i dentro de y .

Si y está vacío devuelve 0, si las muestras pertenecen a la misma clase entonces $Gini = 0$ indicando que es un nodo puro, y mientras más mezclado está menos puro es el nodo y más grande se vuelve la impureza de Gini.

Función `mejor_split(X, y)`: busca la combinación (*feature*, *threshold*) que mejor separa los datos.

1. Si el árbol es parte de un Random Forest (`max_features` no es `None`), primero elige al azar un subconjunto de columnas con `np.random.choice` para ese árbol, con la intención de que cada árbol sea distinto entre sí, en caso contrario se usan todas las features para el árbol.
2. Para cada feature del subconjunto, obtiene los valores únicos ordenados y genera los *thresholds* candidatos como el punto medio entre cada par de valores consecutivos.
3. Para cada *threshold*, separa y en y_{izq} (donde $feature \leq threshold$) y y_{der} (donde $feature > threshold$), calcula el Gini de cada lado con la función anterior, y los pondera por el tamaño de cada partición:

```
gini_ponderado = (n_izq / n_total)* gini_izq + (n_der / n_total)*gini_der
```

4. Se queda con el *threshold/feature* que produce el Gini ponderado más bajo de todos los evaluados, y lo regresa junto con ese valor de Gini.

Función `construir_arbol(X, y, profundidad)`: construye el árbol de forma recursiva.

1. Primero revisa si debe crear una **hoja**: si $gini(y) == 0$ (nodo ya puro), si se llegó a `max_depth`, o si quedan menos de `min_samples_split` muestras. En ese caso, la hoja se etiqueta con la clase mayoritaria.
2. Si no, llama a `mejor_split` para obtener el mejor corte. Si no encontró ningún split válido también crea una hoja con la clase mayoritaria.
3. Si sí hay un split válido, separa X y y en las máscaras izquierda/derecha, y se llama a sí misma recursivamente para construir el subárbol izquierdo y el derecho, aumentando profundidad en 1. El nodo resultante guarda el feature, el threshold y sus dos hijos.

Función fit(X, y): solo llama a construir_arbol(X, y) y guarda el resultado en self.root para poder entrenar el árbol.

Función predict_row(row): recibe una sola fila y recorre el árbol desde self.root, en cada nodo compara row[nodo.feature] contra nodo.threshold para decidir si baja por la izquierda o la derecha, hasta llegar a un nodo hoja y regresa su valor.

```
while not nodo.es_hoja(): # type: ignore
    if row[nodo.feature] <= nodo.threshold: #type: ignore
        nodo = nodo.left #type: ignore
    else:
        nodo = nodo.right #type: ignore
return nodo.valor #type: ignore
```

Función imprimir_arbol: recorre el árbol recursivamente e imprime su estructura.

Clase RandomForest

Función bootstrap(X, y): genera una muestra de entrenamiento para un árbol individual. Recorre samples_size = len(X) veces, y en cada iteración elige un índice aleatorio agregando esa fila a la muestra. Como hay reemplazo, algunas filas del dataset original pueden aparecer varias veces y otras ninguna lo que hace que cada árbol del bosque vea datos distintos.

```
for i in range(samples_size):
    random = np.random.randint(0, samples_size)
    sample_X.append(X[random])
    sample_y.append(y[random])
```

Función fit(X, y): construye el bosque completo.

1. Calcula n_subconjunto = sqrt(n_features), que será el número máximo de features sobre el que se va a entrenar cada árbol.
2. Repite n_arboles veces: genera una muestra bootstrap distinta con la función anterior, crea un DecisionTree nuevo, lo entrena con arbol.fit(sample_X, sample_y), y lo agrega a la lista self.bosque.

```
for i in range(self.n_arboles):
    sample_X, sample_y = self.bootstrap(X, y)
    arbol = DecisionTree(min_samples_split=self.min_samples_split,
max_depth=self.max_depth, max_features=n_subconjunto)
    arbol.fit(sample_X, sample_y)

    self.bosque.append(arbol)
```

Esto es lo que hace que el Random Forest sea distinto de simplemente entrenar el mismo árbol varias veces ya que cada árbol ve una muestra bootstrap distinta de las filas, y en cada split, solo puede elegir entre un subconjunto aleatorio de columnas.

Función predict(X): para cada fila de X, le pide una predicción a cada árbol del bosque, cuenta los votos por clase en un diccionario, y se queda con la clase que tiene más votos. Por eso la interfaz pide un número impar de árboles: en un problema binario evita empates exactos.

```
for arbol in self.bosque:
    prediccion = arbol.predict_row(fila)
    votos[prediccion] = votos.get(prediccion, 0) + 1

#La predicción con mas votos es la ganadora
ganador = max(votos, key=lambda k : votos[k])
```

Funciones auxiliares

Función evaluacion(y_real, y_pred) (binaria): recorre y_real y y_pred con un ciclo for y clasifica cada predicción en TP, TN, FP o FN según coincida o no con la etiqueta real. Con esos cuatro conteos calcula manualmente:

```
accuracy = (TP + TN)/(TP + TN + FP + FN) if (TP + TN + FP + FN) > 0 else 0
precision = TP / (TP + FP) if (TP + FP) > 0 else 0
recall = TP / (TP + FN) if (TP + FN) > 0 else 0
specificity = TN / (TN + FP) if (TN + FP) > 0 else 0
f1 = 2*(precision*recall)/(precision + recall) if precision or recall != 0 else 0
```

Función evaluacion_multiclase(y_real, y_pred): para el caso de más de dos clases se calcula el accuracy global ($\text{np.sum}(y_real == y_pred) / \text{total}$) y construye la matriz de confusión completa que muestra cuántas muestras de cada clase real fueron predichas como cada clase.

Validación cruzada

Antes de tocar el conjunto de test, se usa la función k_fold sobre el conjunto de entrenamiento para tener una estimación del desempeño del modelo menos dependiente de un solo split. La función divide los índices en k bloques del mismo tamaño y, en cada yield usa un bloque distinto como test y el resto como entrenamiento, así se entrena y evalúa un Random Forest nuevo k veces, y se promedia el accuracy obtenido en cada fold.

```
for X_ctrain, X_ctest, y_ctrain, y_ctest in k_fold(X_train, y_train, k=5):
    bosque_cv = RandomForest(2, 5, numero_arboles)
```

```

bosque_cv.fit(X_ctrain, y_ctrain)
prediccion_cv = bosque_cv.predict(X_ctest)

correctos = np.sum(y_ctest == prediccion_cv)
total = len(y_ctest)
accuracy = correctos/total

accuracy_k_fold.append(accuracy)
# Accuracy de cada fold del cross validation
print(f"fold {fold}: accuracy: {accuracy}")
fold += 1
#Promedio de exactitud del cross validation
promedio_cv = np.mean(accuracy_k_fold)
print(f"Cross validation mean accuracy: {promedio_cv: .4f} \n")

```

Metodología

- Carga del dataset a X y Y.
- Partición train/test: partición aleatoria con semilla fija para poder reproducir resultados, usando el 30% del dataset total para test.
- Cross-validation (k_fold, k=5) sobre el conjunto de entrenamiento, entrenando un Random Forest nuevo en cada fold para estimar el accuracy promedio antes de probar con el test set global.
- Entrenamiento final del Random Forest y de un árbol individual sobre el conjunto de entrenamiento completo.
- Evaluación sobre el conjunto de prueba, reportando matriz de confusión y métricas.

Resultados

Dataset Iris binario: Número de árboles, 15, max_depth = 5, min_samples_split = 2, k_fold = 5

Bosque

```
Ingresar el numero de arboles impar: 15
-- BOSQUE --

CROSS VALIDATION --

fold 1: accuracy: 0.9285714285714286
fold 2: accuracy: 0.9285714285714286
fold 3: accuracy: 0.9285714285714286
fold 4: accuracy: 0.9285714285714286
fold 5: accuracy: 1.0
Cross validation mean accuracy: 0.9429

ENTRENAMIENTO--

Predicciones del bosque: [0 0 1 0 1 1 1 0 1 1 1 0 0 0 0 1 0 0 0
1 1 1 1 1 1 1 1 0 1 1]
Valores reales: [1 0 1 0 1 1 1 1 0 1 1 1 1 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 0 1 1]
Accuracy : 0.9000
Acertados: 27 de 30

--Matriz de Confusion--
Prediccion    0    1
Real
0              10    1
1              2    17
-----
```

Arbol

```
-- ARBOL --

feature 3 <= threshold 1.7500
|-- SI - feature 2 <= threshold 4.9500
|   |-- SI - feature 3 <= threshold 1.6500
|   |   |-- SI - Hoja: 0
|   |   |-- NO - Hoja: 1
|   |-- NO - feature 3 <= threshold 1.5500
|   |   |-- SI - Hoja: 1
|   |   |-- NO - Hoja: 0
|   |-- NO - Hoja: 1

Predicciones de un solo arbol:  [0 0 1 0 1 1 1 0 1 1 1 1 0 0 0 1
0 0 0 1 1 1 1 1 1 1 0 1 1]
Valores reales:                  [1 0 1 0 1 1 1 0 1 1 1 1 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 0 1 1]
Accuracy : 0.9333
Acertados: 28 de 30

--Matriz de Confusion--
Prediccion  0  1
Real
0           10  1
1           1  18
-----
```

Dataset Breast Cancer Wisconsin: Número de árboles = 15, max_depth = 5,
min_samples_split = 2, k_fold = 5

Bosque:

```
-- BOSQUE --

CROSS VALIDATION --

fold 1: accuracy: 0.9746835443037974
fold 2: accuracy: 0.9240506329113924
fold 3: accuracy: 0.9367088607594937
fold 4: accuracy: 0.9620253164556962
fold 5: accuracy: 0.8987341772151899
Cross validation mean accuracy: 0.9392

ENTRENAMIENTO--

Predicciones del bosque: [0 1 1 1 0 1 0 0 1 1 1 0 0 0 0 0 0 0 1 0 0 0 1 0 0 1 0 0 1 0 1 0 1 1 0 1 0 0 0 1 0 0 1 1 1 0 0 0 0 1 1 0 1 1 0 0 0 1 0 1 1 1 0 1 0 1
0 0 1 1 1 1 0 1 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0 1 1 0 1 1 0 0 0 1 0 1 1 1 0 1 0 1
0 0 1 1 1 1 0 1 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0 1 0 1 0 0 0 0 0 1 1 1 0 0 1
0 0 1 1 1 1 0 0 1 1 0 0 0 1 1 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0
0 0 0 0 0 1 1 0 0 1 1 1 0 1 1 0 1 0 0 0 0 0 1 0]
Valores reales: [0 1 1 1 0 1 0 0 1 1 1 0 0 0 0 0 0 0 1 0 0 0 1 0 0 1 0 0 1 1 0 1 0 1 0 1 1 1 0 0 1
1 0 1 1 0 0 1 0 1 0 0 1 0 0 1 1 1 0 0 0 1 1 0 1 1 0 0 0 1 0 1 1 1 1 0 0 1
0 0 1 1 1 1 1 1 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0 1 0 1 0 0 0 0 0 1 1 1 0 0 1
0 0 1 1 1 1 0 0 1 1 0 0 0 1 1 1 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0
0 0 0 1 0 0 1 0 0 1 1 1 0 1 1 0 1 0 0 0 0 1 0]
Accuracy : 0.9532
Acertados: 163 de 171

--Matriz de Confusion--
Prediccion  0  1
Real
0           97  2
1           6  66
-----
```

Arbol:

```
-- ARBOL --

feature 23 <= threshold 884.7500
|-- SI - feature 27 <= threshold 0.1603
| | |-- SI - feature 27 <= threshold 0.1323
| | | |-- SI - feature 10 <= threshold 1.0475
| | | | |-- SI - feature 13 <= threshold 38.3500
| | | | | |-- SI - Hoja: 0
| | | | | |-- NO - Hoja: 0
| | | | | |-- NO - Hoja: 1
| | | | |-- NO - feature 1 <= threshold 20.9700
| | | | | |-- SI - feature 23 <= threshold 811.1000
| | | | | | |-- SI - Hoja: 0
| | | | | | |-- NO - Hoja: 1
| | | | | |-- NO - Hoja: 1
| | | |-- NO - feature 21 <= threshold 23.4700
| | | | |-- SI - Hoja: 0
| | | | |-- NO - Hoja: 1
| | |-- NO - feature 26 <= threshold 0.1981
| | | |-- SI - feature 1 <= threshold 21.2600
| | | | |-- SI - Hoja: 0
| | | | |-- NO - Hoja: 1
| | |-- NO - feature 8 <= threshold 0.1394
| | | |-- SI - Hoja: 0
| | | |-- NO - Hoja: 1
```

Accuracy : 0.9415
Acertados: 161 de 171

```
--Matriz de Confusion--
Prediccion  0  1
Real
0           95  4
1           6  66
-----
```


Dataset Wine: Número de árboles = 101, max_depth = 5, min_samples_split = 2, k_fold = 5

Bosque

```
Ingresa el numero de arboles impar: 101
-- BOSQUE --

CROSS VALIDATION --

fold 1: accuracy: 0.9583333333333334
fold 2: accuracy: 0.9583333333333334
fold 3: accuracy: 0.9583333333333334
fold 4: accuracy: 1.0
fold 5: accuracy: 0.9583333333333334
Cross validation mean accuracy: 0.9667
```

```
Accuracy : 0.9815
Acertados: 53 de 54

--Matriz de Confusion--
Prediccion  1  2  3
Real
1           15  0  0
2            0 21  1
3            0  0 17
-----
```

Arbol

```
-- ARBOL --

feature 12 <= threshold 755.0000
|-- SI - feature 6 <= threshold 1.2250
|  |-- SI - feature 9 <= threshold 3.8100
|  |  |-- SI - Hoja: 2
|  |  |-- NO - Hoja: 3
|  |-- NO - feature 9 <= threshold 7.1750
|  |  |-- SI - feature 2 <= threshold 2.8250
|  |  |  |-- SI - Hoja: 2
|  |  |  |-- NO - feature 0 <= threshold 12.5000
|  |  |    |-- SI - Hoja: 2
|  |  |    |-- NO - Hoja: 1
|  |  |-- NO - Hoja: 3
|  |-- NO - feature 11 <= threshold 2.6400
|  |  |-- SI - feature 1 <= threshold 2.0400
|  |  |  |-- SI - Hoja: 2
|  |  |  |-- NO - Hoja: 3
|  |  |-- NO - feature 3 <= threshold 27.5000
|  |    |-- SI - Hoja: 1
|  |    |-- NO - Hoja: 2
```

```
Accuracy : 0.8889
Acertados: 48 de 54

--Matriz de Confusion--
Prediccion  1  2  3
Real
1           12  3  0
2            1 21  0
3            0  2 15
```

Dataset MAGIC Gamma Telescope: Número de árboles = 15, max_depth = 5,
min_samples_split = 2, k_fold = 5

Bosque

```
Ingresa el numero de arboles impar: 15
```

```
-- BOSQUE --
```

```
CROSS VALIDATION --
```

```
fold 1: accuracy: 0.8253193087903832
```

```
fold 2: accuracy: 0.8313298271975957
```

```
fold 3: accuracy: 0.8302028549962435
```

```
fold 4: accuracy: 0.8354620586025545
```

```
fold 5: accuracy: 0.821938392186326
```

```
Cross validation mean accuracy: 0.8289
```

```
ENTRENAMIENTO--
```

```
Predicciones del bosque: [1 1 1 ... 1 1 0]
```

```
Valores reales: [0 1 1 ... 1 1 0]
```

```
Accuracy : 0.8149
```

```
Acertados: 4650 de 5706
```

```
--Matriz de Confusion--
```

```
Prediccion    0    1
```

```
Real
```

```
0             1092   932
```

```
1             124   3558
```

```
-----
```

Arbol

```
Predicciones de un solo arbol: [1 1 1 ... 1 1 0]
```

```
Valores reales: [0 1 1 ... 1 1 0]
```

```
Accuracy : 0.8202
```

```
Acertados: 4680 de 5706
```

```
--Matriz de Confusion--
```

```
Prediccion    0    1
```

```
Real
```

```
0             1266   758
```

```
1             268   3414
```

```
-----
```

Análisis y conclusiones

Tabla Comparativa

Dataset	# Clases	Train / Test	CV accuracy (5-fold)	Accuracy árbol (test)	Accuracy bosque (test)
Iris	2	70 / 30	0.9429	0.9333	0.9000
Wisconsin	2	398 / 171	0.9392	0.9415	0.9532
Wine	3	124 / 54	0.9667	0.8889	0.9815
MAGIC	2	13,314 / 5,706	0.8289	0.8202	0.8149

Algo que me llamó la atención es que el Random Forest no siempre le ganó al árbol solo, en Wine sí hizo diferencia clara (98% vs 89%) y en Wisconsin un poco (95.3% vs 94.1%), pero en Iris y MAGIC el árbol individual quedó un poco arriba.

El dataset de Magic fue el más raro: accuracy parecido entre árbol y bosque, pero las matrices de confusión salen distintas. El bosque tiene más recall (96.6% vs 92.7%) pero mucha menos especificidad (54% vs 62.5%), o sea que atrapa más positivos pero también tira más falsas alarmas.

En Cross Validation vs Test, Wisconsin, Wine y Magic salieron consistentes, lo cual me da confianza de que el modelo generaliza bien ahí. Iris fue el más inconsistente (CV dio 94.3%, hasta un fold en 100%, pero el test cayó a 90%) con 20 muestras de train por fold y 30 de test.

Por último, lo del accuracy más bajo en Magic (81-82%) considero que es más por los hiperparámetros que le pude meter que por el dataset en sí. Es el dataset más grande y por tiempo de cómputo solo le puse max_depth=5 y 15 árboles debido a que tarda mucho en correr, y con esas limitantes ni el árbol ni el bosque alcanzaron a explotar los datos.